```
[2]: import pandas as pd
     import numpy as np
```

```
[3]: email = 'burger@gmail.com'
```

```
[4]: email.split('@')
```

```
[4]: ['burger', 'gmail.com']
```

```
[6]: names = pd.Series(['andrew','bobo','claire','david','5'])
```

```
[7]: names
```

```
[7]: 0    andrew
     1      bobo
     2    claire
     3     david
     4         5
     dtype: str
```

```
[10]: names.str.upper()
```

```
[10]: 0    ANDREW
      1      BOBO
      2    CLAIRE
      3     DAVID
      4         5
      dtype: str
```

```
[12]: email.isdigit()
```

```
[12]: False
```

```
[14]: '5'.isdigit()
```

```
[14]: True
```

```
[15]: names.str.isdigit()
```

```
[15]: 0    False
      1    False
      2    False
      3    False
      4     True
      dtype: bool
```

`.isdigit()` checks whether every character in the string is a digit (0–9). It does not check the data type.

```
[19]: tech_finance = ['GOOG,APPL,AMZN', 'JPM,BAC,GS']
```

```
[20]: len(tech_finance)
```

```
[20]: 2
```

```
[21]: tickets = pd.Series(tech_finance)
```

```
[22]: tickets
```

```
[22]: 0    GOOG,APPL,AMZN
      1        JPM,BAC,GS
      dtype: str
```

```
[23]: tickets.str.split(',')
```

```
[23]: 0    [GOOG, APPL, AMZN]
      1        [JPM, BAC, GS]
      dtype: object
```

```
[25]: tickets.str[0][0]
```

```
[25]: 'G'
```

```
[26]: tech = 'GOOG,APPL,AMZN'
```

```
[31]: tech.split(',')
```

```
[31]: ['GOOG', 'APPL', 'AMZN']
```

```
[29]: tech[0]
```

```
[29]: 'G'
```

```
[32]: tech.split(',')[0]
```

```
[32]: 'GOOG'
```

`tech[0]` returns 'G' because it indexes the original string, whereas `tech.split(',')[0]` first converts the string into a list of words (['GOOG', 'APPL', 'AMZN']) and then returns the first element, 'GOOG'.

```
[33]: tickets.str.split(',').str[0]
```

```
[33]: 0    GOOG
      1     JPM
      dtype: object
```

Suppose I want to make tickets into three columns:

```
[34]: tickets.str.split(',',expand=True)
```

[34]:

	0	1	2
0	GOOG	APPL	AMZN
1	JPM	BAC	GS

`expand=True` tells Pandas to expand each split list into separate columns, and since multiple columns are created, the result is a DataFrame instead of a Series.

```
[37]: messy_names = pd.Series(['andrew     ','bo;bo','    claire    '])
```

```
[38]: messy_names
```

```
[38]: 0       andrew
      1            bo;bo
      2        claire
      dtype: str
```

```
[39]: messy_names.str.replace(';','')
```

```
[39]: 0       andrew
      1             bobo
      2        claire
      dtype: str
```

Now, to remove unnecessary spaces:

```
[42]: messy_names.str.replace(';','').str.strip().str.capitalize()
```

```
[42]: 0    Andrew
      1      Bobo
      2    Claire
      dtype: str
```

If anything is hard to do, just create a custom function and use `apply()`

```
[43]: def cleanup(name):
          name = name.replace(';','')
          name = name.strip()
          name = name.capitalize()
          return name
```

```
[44]: messy_names.apply(cleanup)
```

```
[44]: 0    Andrew
      1      Bobo
      2    Claire
      dtype: str
```